UNIVERSIDAD AUTÓNOMA

FACULTAD DE CIENCIAS

LICENCIATURA EN CIENCIAS COMPUTACIONALES

MANUAL DE PRÁCTICAS
ASIGNATURA: 6.3 AUTÓMATAS Y COMPILADORES

---

fondo.png

Práctica No. 4: Análisis léxico de palabras

Nombre de la práctica: Análisis léxico de palabras

No. de práctica: 4 — **No. de sesiones:** 2

No. de integrantes máximo por equipo: 2

Profesor:
Eduardo Cornejo

Integrante:
Daniela López Martínez

Fecha de entrega: 8 de octubre de 2026

1. Introducción

El analizador léxico es la primera fase de un compilador. Su principal función consiste en leer los caracteres de entrada y elaborar como salida una secuencia de componentes léxicos que utiliza el analizador sintáctico para hacer su análisis.

El análisis léxico también es llamado exploración o *scanner*. Su función es leer caracteres uno a uno desde la entrada e ir formando grupos de caracteres con alguna relación entre sí llamados *tokens*, los que serán la entrada para la siguiente etapa del compilador.

El analizador léxico:

- Es la primera fase del programa traductor.
- Es el único que gestiona el archivo de entrada con el código fuente.
- Es el que lee los caracteres del programa fuente y construye símbolos intermedios, los cuales serán la entrada del analizador sintáctico.

El analizador léxico típicamente detecta los siguientes errores:

- Utilizar caracteres que no pertenecen al alfabeto del lenguaje.
- Cadenas que no coinciden con ninguno de los patrones de los tokens posibles.

2. Objetivo General

Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

3. Objetivos Específicos

⋆ Identificar el proceso básico de análisis de cadenas de símbolos de un alfabeto.

⋆ Implementar estructuras de datos y algoritmos de análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

⋆ Implementar expresiones regulares para el análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

4. Equipo y Software a Utilizar

Equipo de cómputo con software para programación a elección de los alumnos (Flex / Lex, GCC / CC, EditPlus o Editor de Texto).

5. Procedimiento para el Desarrollo de la Práctica

Para la compilación e implementación desde línea de comandos se utilizan las siguientes instrucciones:

```
C:\>lex [nombre-programa].l
C:\>cc -w lex.yy.c -o [nombre-ejecutable]
C:\>[nombre-ejecutable].exe
```

6. Desarrollo y Resultados (Programas en Lex)

6.1. Programa 1: Reconocer números enteros

```
%{
#include <stdio.h>
%}

%%
[0-9]+       { printf("TOKEN: Numero Entero -> %s\n", yytext); }
[ \t\n]+     ; /* Ignorar espacios */
.            { printf("Caracter no valido -> %s\n", yytext); }
%%

int yywrap() { return 1; }

int main() {
    printf("--- Reconocedor de Numeros Enteros ---\n");
    yylex();
    return 0;
}
```

programa1.l

[Pegar aquí la captura de pantalla del Programa 1]

Figura 1: Ejecución y prueba de reconocimiento de enteros.

6.2. Programa 2: Reconocer números decimales

```
%{
#include <stdio.h>
%}

%%
[0-9]+"."[0-9]+   { printf("TOKEN: Numero Decimal -> %s\n", yytext); }
[ \t\n]+          ; /* Ignorar espacios y saltos de linea */
.                 { printf("Caracter no valido -> %s\n", yytext); }
%%

int yywrap() { return 1; }

int main() {
    printf("--- Reconocedor de Numeros Decimales ---\n");
```

```
15      yylex();
16      return 0;
17  }
```

programa2.l

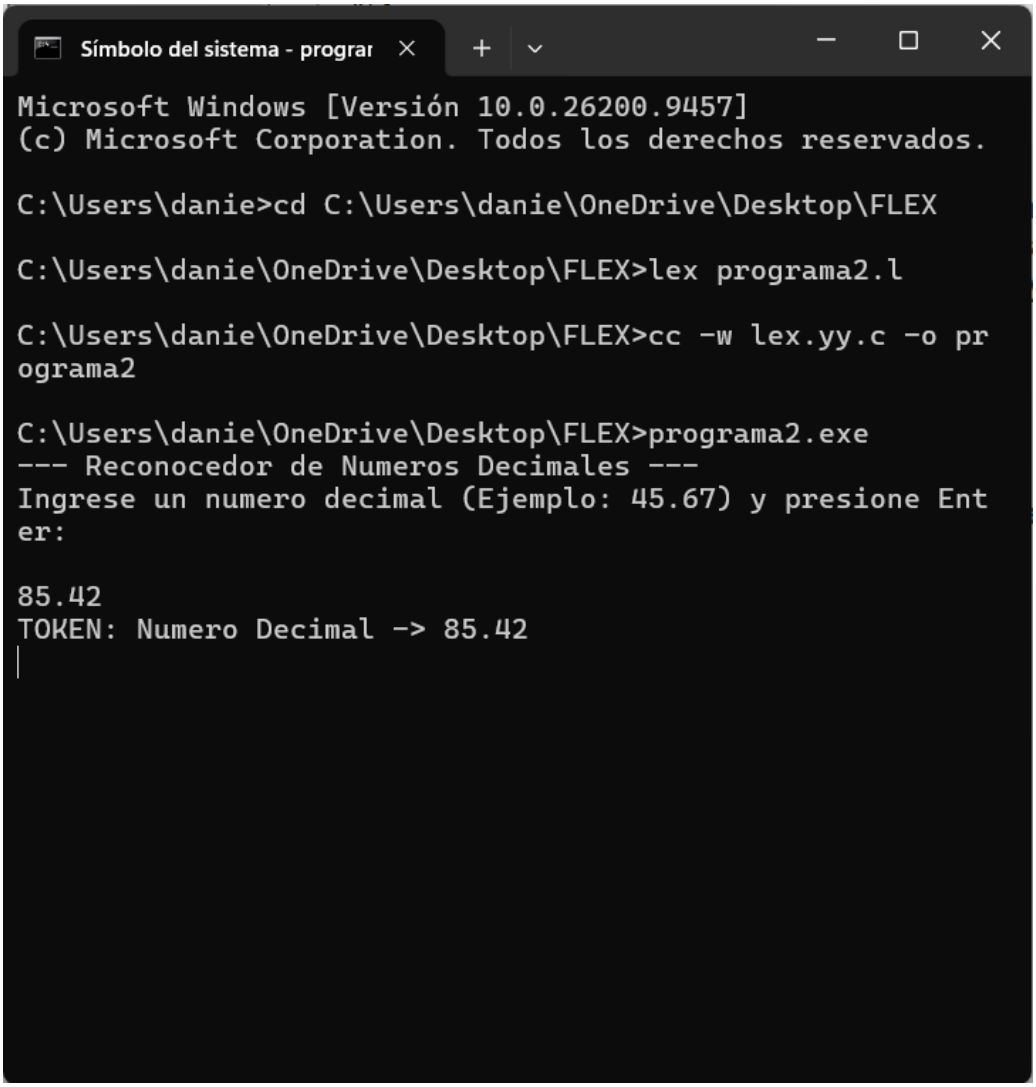


Figura 2: Ejecución y prueba de reconocimiento de decimales.

6.3. Programa 3: Reconocer variables (identificadores)

```
%{
#include <stdio.h>
%}

%%
[a-zA-Z_][a-zA-Z0-9_]*    { printf("TOKEN: Identificador/Variable -> %s\n", yytext); }
[ \t\n]+                  ; /* Ignorar espacios y saltos de linea */
.                         { printf("Caracter no valido -> %s\n", yytext); }
%%

int yywrap() { return 1; }

int main() {
    printf("--- Reconocedor de Variables / Identificadores ---\n");
    yylex();
    return 0;
}
```

programa3.l

6.4. Programa 4: Reconocer RFC

```
%{
#include <stdio.h>
%}

%%
[A-Z &]{3,4}[0-9]{6}[A-Z0-9]{3}   { printf("TOKEN: RFC Valido -> %s\n", yytext); }
[ \t\n]+                          ; /* Ignorar espacios y saltos de linea */
.                                 { printf("Caracter no valido -> %s\n", yytext); }
%%

int yywrap() { return 1; }

int main() {
    printf("--- Reconocedor de RFC ---\n");
    yylex();
    return 0;
}
```

programa4.l

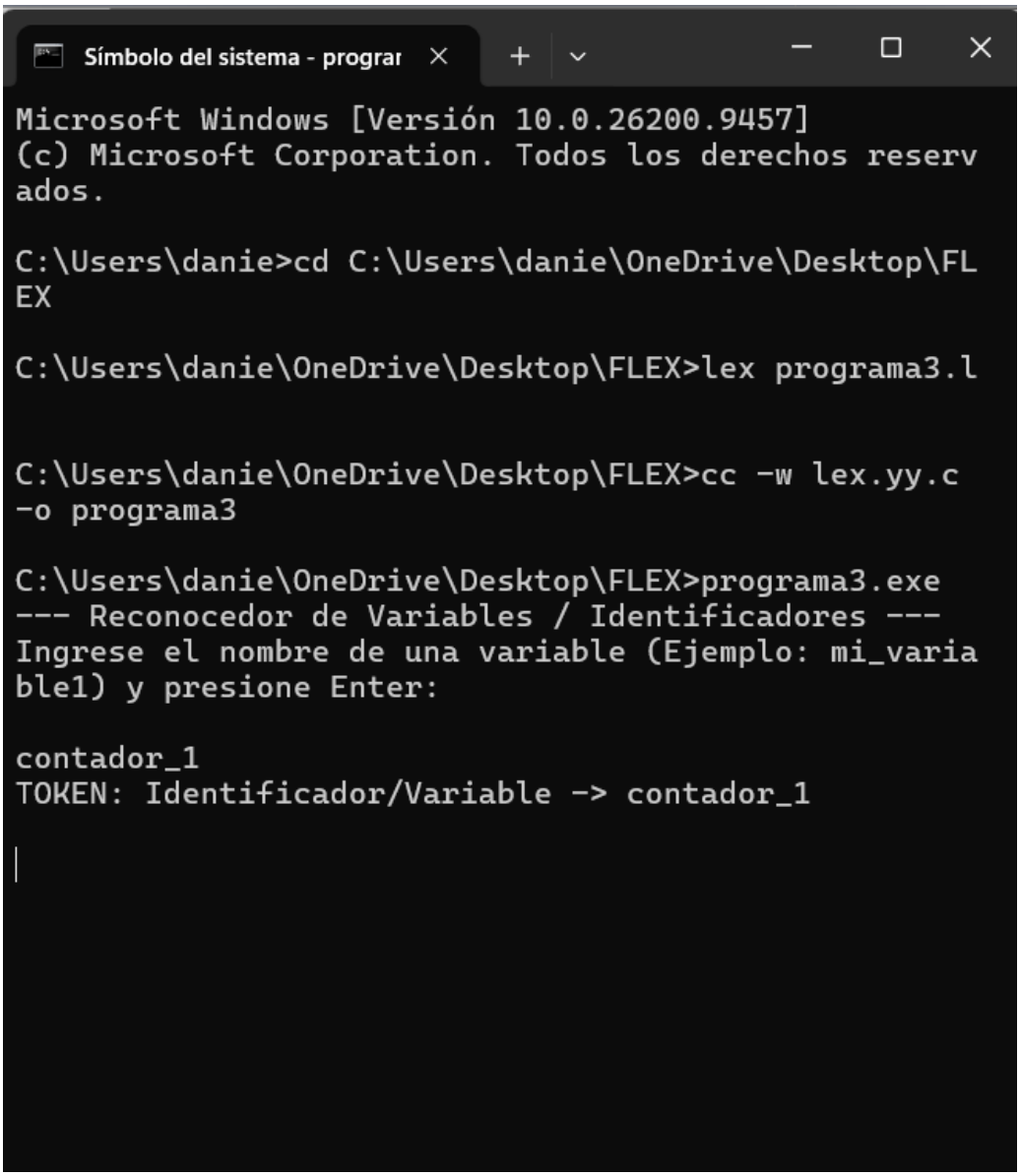


Figura 3: Ejecución y prueba de reconocimiento de variables.

6.5. Programa 5: Reconocer CURP

```
%{
#include <stdio.h>
%}

%%
[A-Z][AEIOUX][A-Z]{2}[0-9]{6}[HM][A-Z]{5}[0-9A-Z]{2}  { printf("TOKEN: CURP Valida -> %s\n", yytext); }
[ \t\n]+                                               ; /* Ignorar espacios y saltos de linea */
.                                                      { printf("Caracter no valido -> %s\n", yytext); }
%%

int yywrap() { return 1; }

```

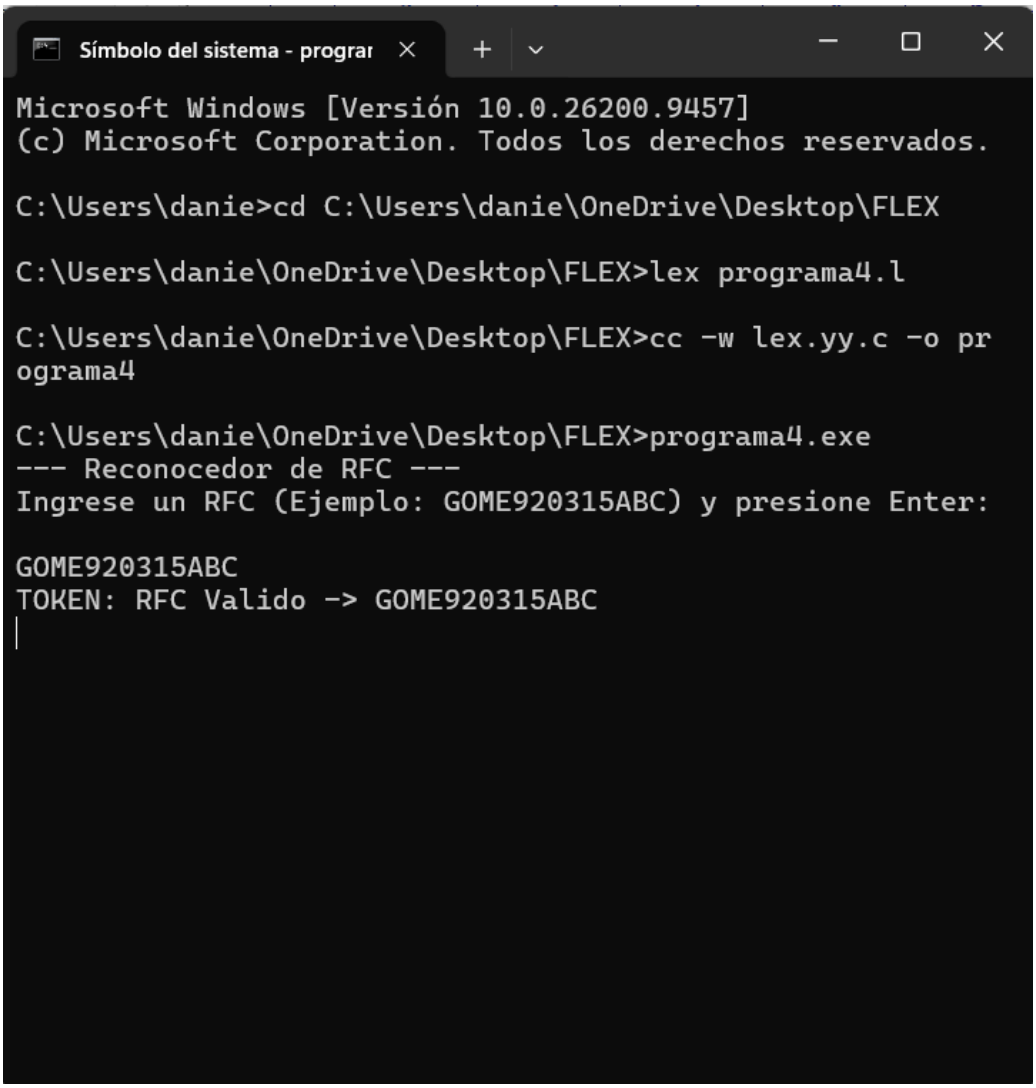


Figura 4: Ejecución y prueba de reconocimiento de RFC.

```
int main() {
    printf("--- Reconocedor de CURP ---\n");
    yylex();
    return 0;
}
```

programa5.l

6.6. Programa 6: Reconocer email institucional

```
%{
#include <stdio.h>
%}

%%
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.(edu|edu\.[a-z]{2}|mx)    { printf("
    TOKEN: Email Institucional -> %s\n", yytext); }
[ \t\n]+                                                    ; /* Ignorar
    espacios y saltos de linea */
```

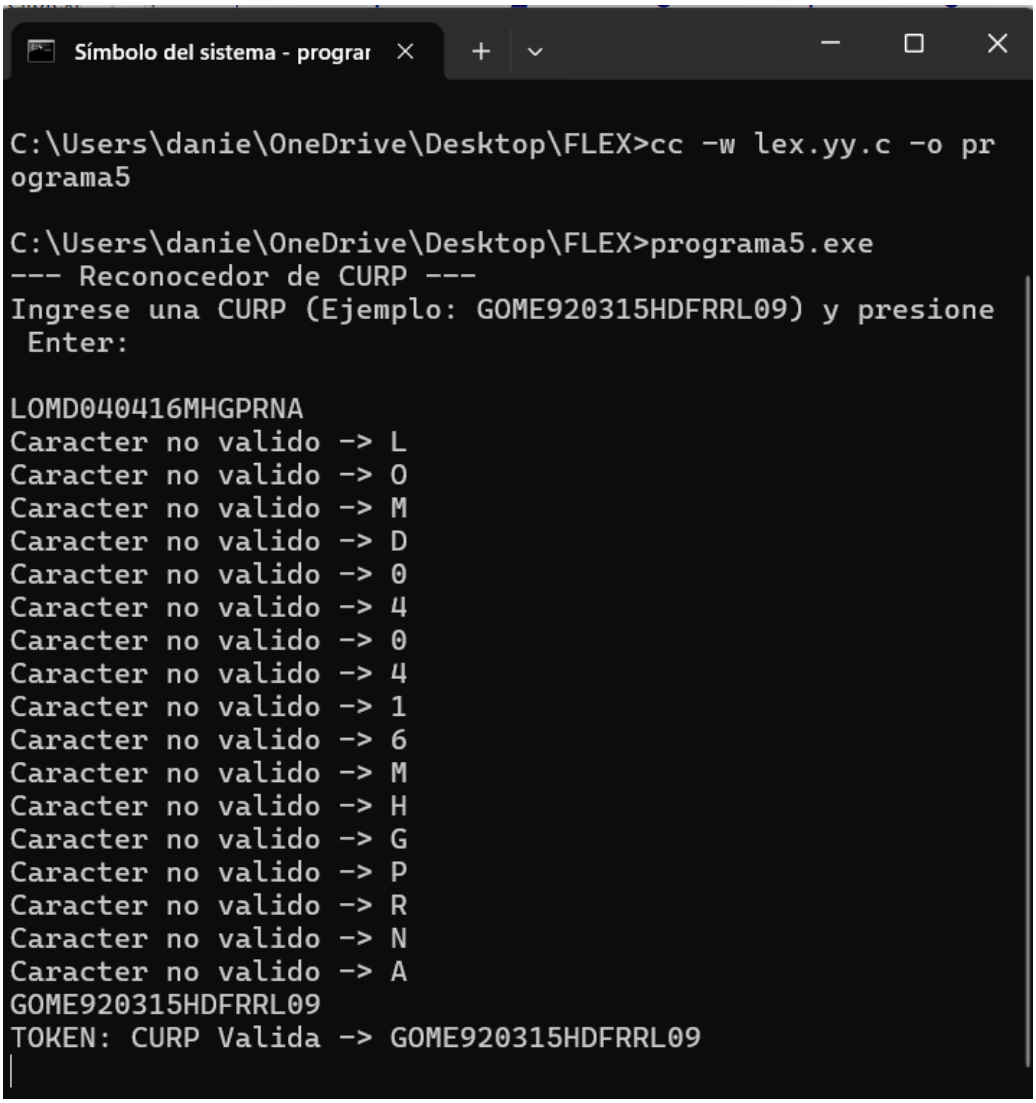


Figura 5: Ejecución y prueba de reconocimiento de CURP.

```
8  .                                                    { printf("
       Caracter no valido -> %s\n", yytext); }
9  %%
10
11 int yywrap() { return 1; }
12
13 int main() {
14     printf("--- Reconocedor de Email Institucional ---\n");
15     yylex();
16     return 0;
17 }
```

programa6.l

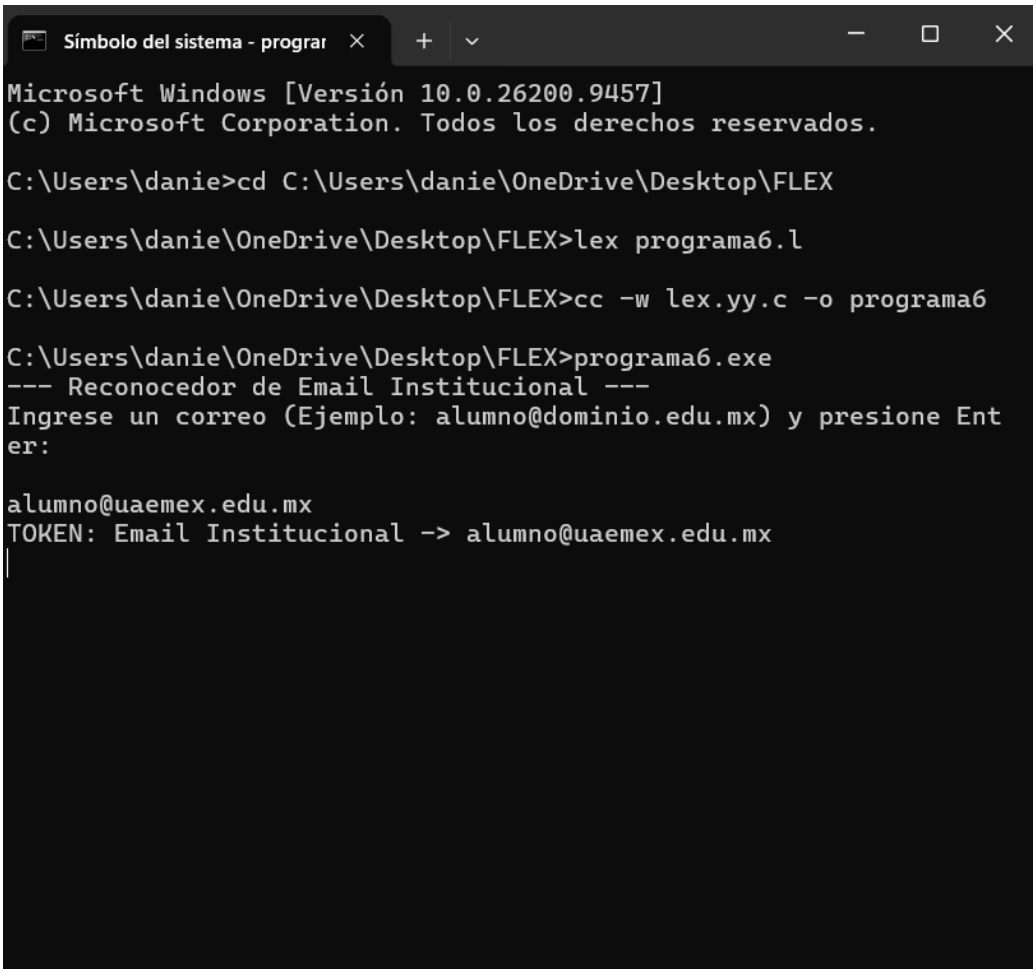


Figura 6: Ejecución y prueba de reconocimiento de Email Institucional.

7. Cuestionario

1. **¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?**

   Las expresiones regulares permiten definir de forma compacta, formal y precisa los patrones sintácticos que representan las palabras o tokens de un lenguaje. Su principal utilidad radica en que sirven como especificación matemática para que herramientas automatizadas (como Lex/Flex) construyan autómatas finitos deterministas capaces de reconocer lenguajes regulares con alta eficiencia computacional.

2. **¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste?**

   En el entorno de desarrollo Lex se utilizaron las siguientes estructuras y variables:

   - **Variable ‘yytext‘:** Cadena de caracteres (`char*`) que almacena dinámicamente el lexema actual que ha hecho coincidencia con la regla.
   - **Reglas de traducción:** Mapeo de patrones (expresiones regulares) asociados a acciones en C (`printf`) para categorizar las salidas.
   - **Función ‘yylex()‘ y ‘yywrap()‘:** Estructuras de control para el manejo del flujo de lectura del archivo de entrada o consola.

3. ¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?

a) Análisis previo del patrón deseado (extracción de longitud, caracteres alfabéticos, numéricos y especiales).

b) Definición de la expresión regular equivalente.

c) Lectura carácter por carácter desde la entrada estándar vía la herramienta Lex.

d) Evaluación de la coincidencia del token utilizando el principio de la coincidencia más larga (*longest match*).

e) Emisión de un mensaje clasificatorio cuando la coincidencia es exitosa o la señalización de un error léxico ante caracteres no reconocidos.

4. ¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?

Elementos utilizados: Metacaracteres como corchetes `[]`, cuantificadores de repetición (`+`, `*`, {`n,m`}), rangos alfanuméricos `[a-z]`, `[0-9]` y el operador or/agrupamiento.

Recomendaciones y condiciones a seguir:

- **Prioridad de reglas:** Colocar patrones más específicos primero (como CURP o RFC) antes de patrones más generales (como identificadores de variables).
- **Escapar caracteres especiales:** Utilizar comillas o diagonales inversas para símbolos reservados como la barra punto `"."`.
- **Tratamiento de errores:** Definir una regla por defecto al final (`.`) para capturar cualquier símbolo fuera del alfabeto del lenguaje.

8. Bibliografía

- Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
- Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
- Brookshear, J. G. (1995). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.